



Test for BackEnd Engineer

Instructions

1. Attempting all the questions is not mandatory.
2. **Partially complete solutions are welcome.** We want to see your approach and understanding.
3. Once the task is completed, please **share output on email**

High-Level Design Test

Objective:

Design a system to support a scalable, cost-efficient, and performant backend for a machine translation system.

Questions:

1. System Architecture:

- Propose an architecture to handle a machine translation service. Your architecture should address the following:
 - Compute resources for hosting and running the model (GPU/CPU selection and deployment approach).
 - Load balancing for handling translation requests.
 - Database or storage strategy for managing user data and translation metadata.

2. Scalability:

- Explain how your system will scale horizontally or vertically to handle increased user loads.
- Detail how the system will handle idle periods to reduce costs.
- Propose a caching strategy to optimize frequent translation requests.

3. Performance Requirements:

- Explain how you will meet a target of 1,500 words per minute translation speed.
- Discuss strategies to achieve translating a 15,000-word document within 12 minutes during peak load.
- Describe how you will manage GPU/CPU allocation to optimize compute usage.
- Detail strategies for pooling resources during idle periods to minimize costs
- Highlight considerations for maintaining performance with large model sizes (60–100GB).

4. Priority Handling:

- Design a mechanism for prioritizing high-priority translation requests.
- Provide a queue-based or request-tagging solution that ensures priority jobs are processed immediately.

5. Monitoring and Cost Estimates:

- Detail a monitoring strategy for tracking system health, model performance, and resource usage.
- Provide cost estimates for handling 10,000, 100,000, and 1 million words per day, with assumptions on compute usage, TFLOPS, and storage costs.
- Integrate a monitoring tool to log metrics like:
 - GPU/CPU utilization.
 - API request latency.
 - Queue processing times.
- Write a sample log entry and show how alerts can be triggered for unusual behavior.

Low-Level Design Test

Objective:

Focus on the implementation details of the backend system.

Questions:

1. API Design:

- Design RESTful or gRPC APIs for the following use cases:
 - Submitting a translation request.
 - Fetching the status of a translation job.
 - Fetching the translated document.
- Include considerations for rate limiting, authentication, and request validation.

2. Database Schema Design:

- Provide a database schema for:
 - Tracking user requests and associated metadata.
 - Storing translation job statuses.
 - Maintaining priority queues or tags for urgent jobs.

3. Code Implementation:

- Write pseudocode for:
 - Processing translation requests using the task queue.
 - Scaling logic that triggers new instance creation or shuts down unused instances.